

Microprocessors Laboratory Report

Lab 1: Assembly Programming & UART Communication

Fraidakis Ioannis

April 2025

Executive Summary

This laboratory exercise explores low-level microcontroller programming through three ARM Cortex-M4 assembly language implementations, seamlessly integrated with C for system-level operation. The project focuses on:

- **Character-based Hash Algorithm:** Implementation of a custom string hashing function with character-specific processing logic and then optimize it to achieve a 43.5% performance improvement
- **Recursive Computation:** Implementation of a recursive Fibonacci function
- **Hardware Communication:** Integration with UART peripheral for real-time data exchange between the microcontroller and external systems

The implementations demonstrate critical concepts in embedded systems development, including mixed-language programming, hardware peripheral control, and algorithm optimization techniques specifically designed for resource-constrained environments. Each solution addresses different aspects of microcontroller programming, from computational efficiency to memory management and I/O operations.

Project Structure

The project is organized into a modular structure with dedicated folders for each problem solution and multiple input modes:

Main Entry Points

Multiple C files provide different input methods for testing:

- **hardcoded_input.c** - Input is provided directly in the code
- **UART_input.c** - Receives input via UART hardware interface
- **scanf_input.c** - Uses Keil's serial console for input
- **test_optimization.c** - Performance benchmark harness for Question 1
- **return_ptr.c** - Used to run `fib_return_ptr.s` variant for Question 3

Execution Note: The project is configured to use `hardcoded_input.c` by default. Other entry points can be enabled by uncommenting the desired file and commenting out the currently active one.

Implementation Folders

- **Question_1/** - String hashing implementations:
 - `hash.s` - Basic implementation
 - `hash_optimized.s` - Optimized version with performance improvements
 - `min_registers.s` - Uses only scratch registers (no stack usage)
- **Question_2/** - Digit reduction algorithm:
 - `sum_mod.s` - Straightforward implementation
- **Question_3/** - Fibonacci implementations:
 - `fibonacci.s` - Standard recursive implementation
 - `fib_return_ptr.s` - Returns address of result, stored in a local variable
- **Bonus/** - Contains the solution for the bonus question

Technical Implementation

Core Function Analysis

- **String Hasher (hash_final.s)** - Character processing logic:

```

1      ; Digit conversion
2      LDR R8, =digit_table    ; Load lookup table
3      LDRB R7, [R8, R7]      ; Get table value
4
5      ; Uppercase handling
6      ADDLE R5, R5, R6, LSL #1 ; 2*ASCII via barrel shifter
7
8      ; Lowercase calculation
9      SUB R7, R6, #'a'        ; Offset from 'a'
10     MLA R5, R7, R7, R5      ; (offset)^2 + hash
11

```

Processing Logic:

- *Digits*: Mapped to predefined values (e.g., '0'→5, '1'→12) via `digit_table`
 - *Uppercase*: Add 2×ASCII value via barrel shifter (LSL #1)
 - *Lowercase*: Add (char-'a')² value using MLA instruction (fusing multiply and add)
 - *Special Chars*: Skipped in processing
- **Digit Reducer (sum_mod.s)** – Core reduction logic:

```

1      ; Digit summation
2      MOV R5, #10             ; Set divisor for decimal base
3      UDIV R6, R4, R5         ; R6 = R4 / 10
4      MLS R7, R5, R6, R4      ; R7 = R4 - (R5 * R6) -> extract last digit
5
6      ; Modulo 7 operation
7      MOV R5, #7              ; Set divisor for modulo
8      UDIV R6, R4, R5         ; R6 = R4 / 7
9      MLS R7, R5, R6, R4      ; R7 = R4 - (R5 * R6) -> R4 mod 7

```

Algorithm:

- Extracts each digit of the hash value (e.g., 1234 → 1, 2, 3, 4) by repeatedly dividing by 10 and accumulating the sum (1234 → 1+2+3+4=10).
- Computes modulo 7 of the sum (e.g., 10 % 7 = 3).

Technical Implementation (continued)

Core Function Analysis

- **Fibonacci (fibonacci.s)** - Recursive implementation with stack management:

```

1      ; Base case handling
2      CMP R0, #0          ; Check n=0
3      BXEQ LR             ; Return 0 (R0 is already 0)
4      CMP R0, #1          ; Check n=1
5      BXEQ LR             ; Return 1 (R0 is already 1)
6
7      ; Recursive decomposition
8      PUSH {LR, R0}       ; Save return address and n
9      SUB R0, R0, #1      ; n-1
10     BL fibonacci        ; fib(n-1)
11     POP {R1}            ; Restore n
12     PUSH {R0}           ; Save fib(n-1)
13     SUB R0, R1, #2      ; n-2
14     BL fibonacci        ; fib(n-2)
15     POP {R1}            ; R1 = fib(n-1)
16     ADD R0, R0, R1      ; R0 = fib(n-1) + fib(n-2)
17     POP {PC}            ; Return
18

```

Mechanism: Implements the recursive definition $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$:

- Preserves return address (LR) and registers using PUSH/POP
- Uses R0 for both input parameter and return value
- Uses conditional return for base cases to minimize branching

Testing & Validation

Testing Methodology

To validate our implementations, we performed comprehensive testing using both static test vectors and dynamic inputs. For performance evaluation, we used the Keil μ Vision simulator with "Performance analyzer" enabled to measure execution time of the optimized versus the baseline hash implementation.

Validation Results

Input	Raw Hash	Reduced	Fib	Checksum	Characteristics
" (empty)	0	0	0	0	Edge case
"Keil uVision"	1752	1	1	90	Alphabetic
"STM32-F411RE"	983	6	8	3	Mixed case
"Arm 2025"	601	0	0	123	Alphanumeric

Optimization Techniques

Optimization Strategies

Key optimizations implemented in the hash function:

- **Single-Pass Processing:** Integrated string length calculation during character processing (eliminated separate traversal)
- **Conditional execution:** Implementing uppercase character processing with conditional execution (LE suffix) to avoid branch penalties:

```

1      CMP      R6, #'Z'          ; Compare with 'Z'
2      ADDLE    R5, R5, R6, LSL #1 ; Conditionally execute if <='Z'
3      BLE      loop_start
4

```

- **Barrel shifter utilization:** Employing the ARM barrel shifter for efficient multiplication by powers of 2:

```

1      ADDLE R5, R5, R6, LSL #1 ; Instead of separate shift
2

```

- **Combined operations:** Using MLA (Multiply-Accumulate) to perform multiplication and addition in a single instruction:

```

1      MLA R5, R6, R6, R5 ; R5 = (R6*R6) + R5 in one cycle
2

```

- **Efficient character classification:** Implementing a single-branch lowercase character check:

```

1      SUB r0, r0, #'a'          ; Offset from 'a'
2      CMP r0, #25                ; Check if in range 0-25 ('a' to 'z')
3      BHI not_lowercase          ; Branch if higher (unsigned)
4

```

- **Post-index LDRB:** Used to combine load and increment in one instruction:

```

1      LDRB R6, [R4], #1 ; Instead of separate ADD for R4
2

```

Performance Optimizations: Our optimized hash implementation achieved a **43.5% speedup** over the baseline implementation. The benchmark was performed on a 1000-character string with an equal distribution of digits, uppercase and lowercase letters, and some symbols.

Conclusion & Learning Outcomes

This lab provided valuable experience in several areas of embedded programming:

- **ARM Assembly Mastery:** Gained practical experience with ARM instruction set architecture, including efficient use of specialized instructions.
- **Register Optimization:** Understanding of register allocation constraints, a critical skill for interrupt handlers and resource-constrained environments
- **Optimization Techniques:** Learned to identify and implement performance optimizations for resource-constrained environments, achieving significant speed improvements through instruction selection and branch elimination
- **Mixed-Language Programming:** Developed skill in integrating C and assembly code, including proper parameter passing and return value handling

Future Work

Based on our experience and performance analysis, several optimization opportunities remain for future exploration:

- **Elimination of Division Instructions:** Replace UDIV operations with multiplication-based division for constant divisors (e.g. 10). For example:

```

1      ; Instead of:
2      MOV R5, #10
3      UDIV R6, R4, R5
4
5      ; Division by 10 via reciprocal
6      LDR R5, =0xCCCCCD ; Magic constant for 1/10 in Q32 format
7      UMULL R6, R7, R4, R5 ; R7 = (R4 * magic) >> 32 = R4 / 10
8      MOV R7, R7, LSR #3   ; Complete the division
9

```

Impact on Cortex-M4: While the M4 implements UDIV in hardware (2-12 cycles, divisor-dependent), reciprocal multiplication reduces worst-case execution time (WCET).

- **Loop unrolling:** Apply strategic loop unrolling in the string processing loop to reduce branch prediction overhead for long input strings
- **Register allocation refinement:** Further tune register usage patterns for improved locality and reduced memory operations
- **Hybrid Digital Root:** Combine digit summation with modulo 9 properties (e.g., `digital_root = 1 + (num - 1) % 9`) for single-step digit addition